Malware Forensics (MalFor)

Coursework

REPORT

Student No.: UP2115789
Word Count: 1510

Disclaimer
By submitting this report for marking, **I fully understand** that and **I agree**:
- This is an independent piece of coursework, and it is expected that I have taken responsibility for all the design, implementation, analysis of results and writing of the report. And, it is 1500 words (+/-10%).
- For the Turnitin plagiarism software, marks will be investigated accordingly, if appropriate.
- This marking scheme sets out what would be expected for each marking band for this coursework (on Moodle).
- It is blind marked.
- Any technical help, high-level advice and suggestions which may be provided in-person (e.g., labs) and electronically, has no contribution to or an indication of my final mark.
- Knowledge of a peer's work and their final mark is not justification for what my own mark should be.
- Any examples provided were used for ideas only, and "having followed an example" is not justification for what my mark should be.
- The coursework malware was used for analysis only and was isolated to a safe working environment (a virtual machine), to prevent the malware from infecting the host.

Word Count

Section	Word Count
Malware Details	134
Static Analysis	245
Dynamic Analysis	319
Reverse Engineering	161
Discussion	612
References	39
Total	1510

1. Malware Details

1.1

This report provides a forensic analysis of the malware sample malfor-cw.exe, identified in the Gasgo data theft incident. The investigation's objective is to reverse engineer the sample's capabilities to provide actionable intelligence. Using static, dynamic, and code-level analysis, this report will document the malware's methods for persistence, command and control (C2), and data exfiltration. The findings conclude with specific recommendations to mitigate this threat and harden the client's security posture. Initial inspection revealed the use of the UPX packer, a common obfuscation technique designed to hinder static analysis and evade signature-based detection.The use of the UPX packer is a deliberate obfuscation step by the author, designed to compress the binary and, more importantly, to hinder static analysis and evade basic signature-based antivirus detection. The file's fundamental identifiers are detailed in Table 1.

Figure 1: VirusTotal Analysis.

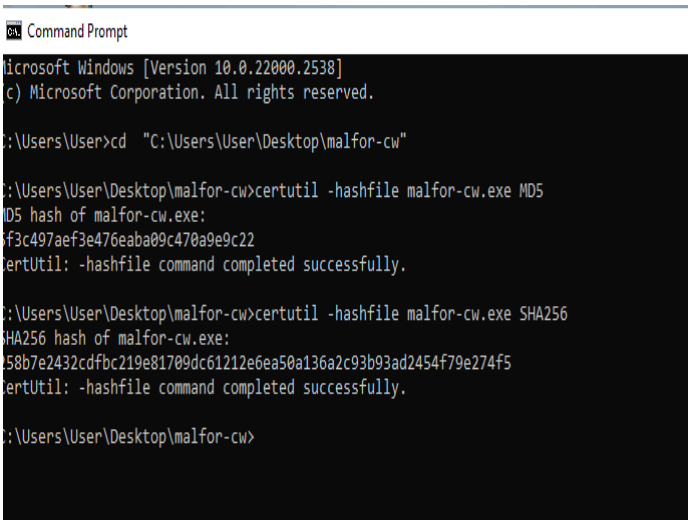


Figure 2: Hash numbers using certutil.

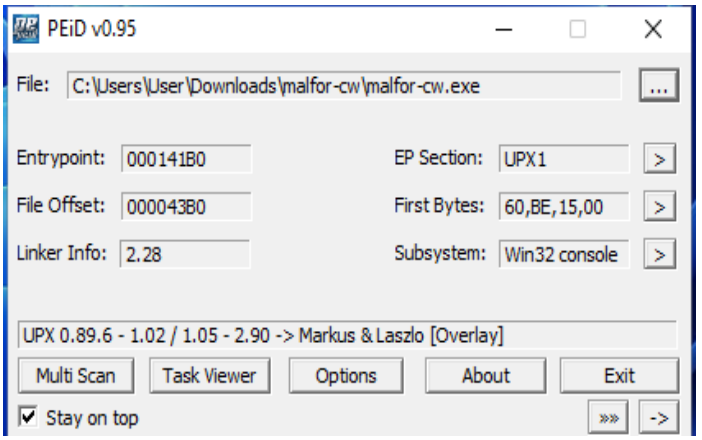


Figure 3: PEiD outcome.

Table 1: Malware Sample details

Attribute	Value
Filename	malfor-cw.exe
MD5 Hash	f3c497aef3e476eaba09c470a9e9c22
SHA256 Hash	258b7e2432cdfbc219e81709dc61212e6ea50a136a2c93ad2454f79e274f5
Compilation Timestamp	2025/11/12 Wed 09:00:55 UTC
Architecture	32-bit (IMAGE_FILE_MACHINE_1386
Packer Detected	UPX 0.89.6-2.90 [Overlay]

2. Static Analysis

Static analysis was performed on the unpacked executable to extract embedded indicators and formulate initial hypothesis about the malware's functionality without executing it.

2.1 String and Import Analysis
Analysis of the unpacked binary strings and imported libraries revealed significant indicators of malicious capability. Key strings pointed towards command and control (C2) communication, persistence, anti-analysis functions.

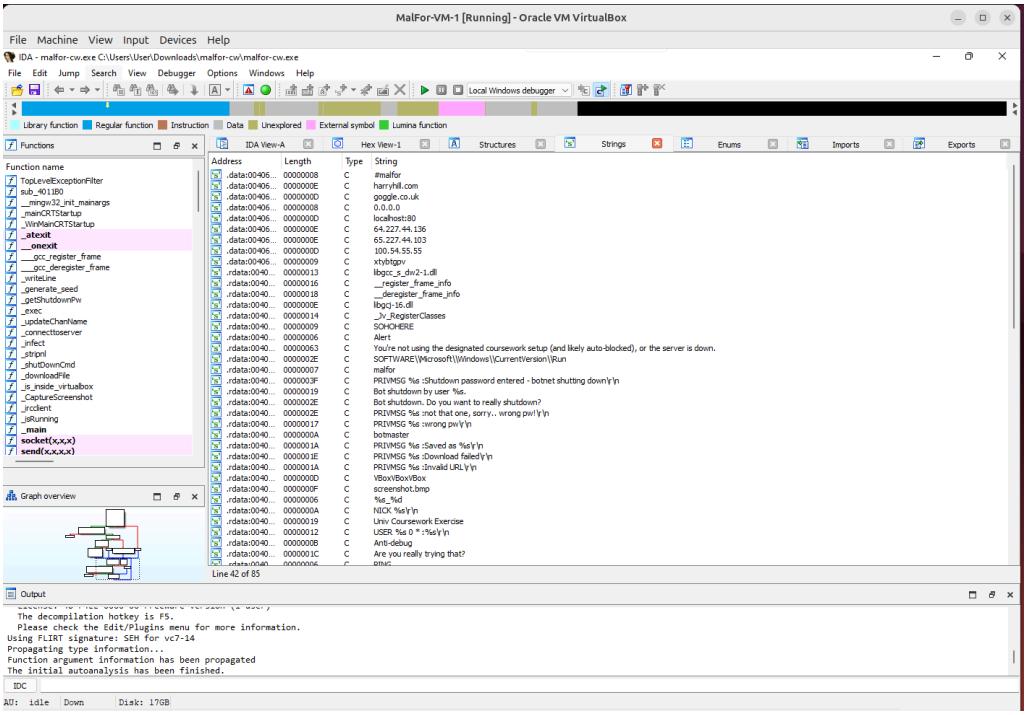


Figure 4: Key strings identified in IDA Pro. Notable indicators include the C2 server IP address, the persistence registry key, and an anti-analysis string for detecting VirtualBox environments.

The import table analysis further supports these hypotheses. The malware imports functions from key Windows libraries, as shown in Figure 4, to support the hypothesis.

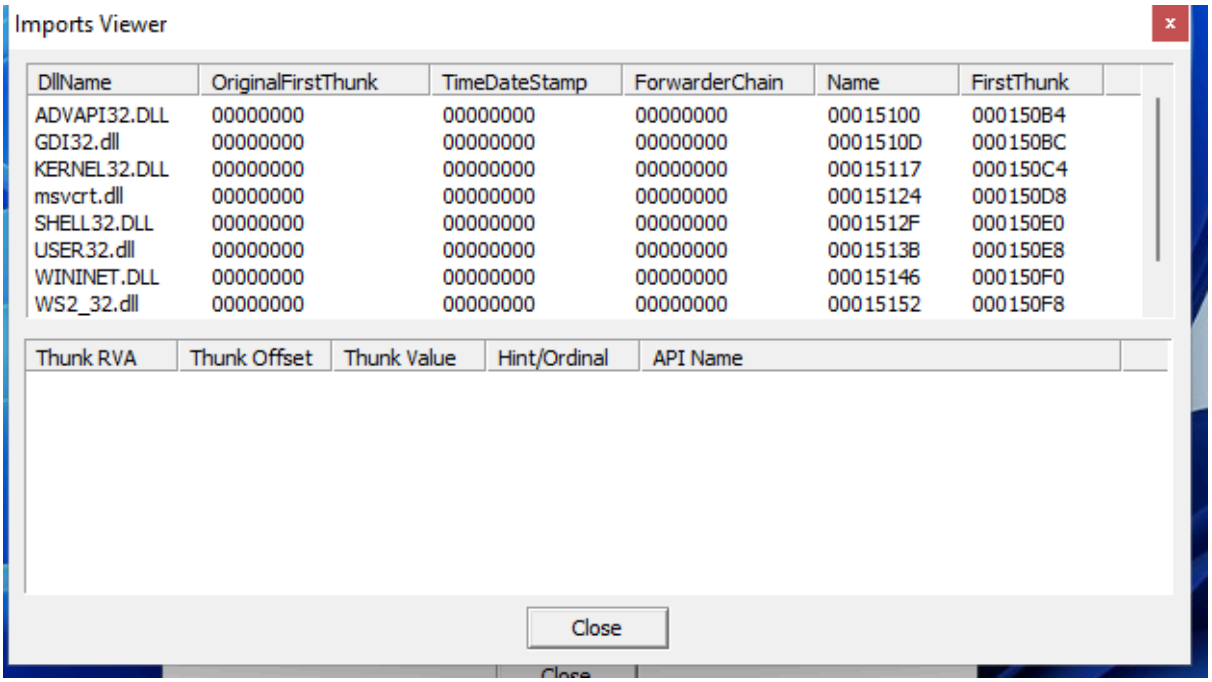


Figure 5: Imported libraries reveal dependencies on WININET.DLL and WS2_32.dll for network communication, and ADVAPI32.DLL for registry manipulation, confirming the potential for C2 and persistence capabilities.

- WININET.DLL and WS2_32.dll are used for network communications.
- ADVAPI32.DLL provides access to the Windows Registry, correlating with the suspected persistence mechanism.
- KERNEL32.DLL contains core functions, including process and memory management, as well as anti-debugging APIs.

The discovery of these indicators during static analysis is crucial, as it allowed for the formation of a detailed and accurate hypothesis about the malware's capabilities—persistence, C2 communication, and anti-analysis—before taking the risk of executing the code. The combination of WININET.DLL for network functions and ADVAPI32.DLL for registry access is a classic pairing for a remote access trojan or bot.

2.2 Initial Hypothesis

Based on the static evidence, the initial hypothesis is that malfor-cw.exe is a backdoor or bot client. It is expected to:

1) Establish persistence by creating a registry key in the CurrentVersion\Run path.
2) Connect to a C2 server at 64.227.44.136 using the IRC protocol.
3) Employ basic virtual machine detection and anti-debugging checks to evade analysis.

3. Dynamic Analysis

To validate the hypothesis from static analysis, the malware was executed in an isolated Windows virtual machine. Network traffic and system changes were monitored to observe its live behaviour.

3.1 Network Communication and C2 Protocol

Upon execution, the malware immediately attempted to establish a TCP connection to the IP address 64.227.44.136 on port 6667, a standard port for IRC. Analysis of the network traffic confirmed the use of IRC protocol.The malware's reliance on a single, hardcoded IP address (64.227.44.136) for C2 communication reveals a critical weakness. This method provides a single point of failure, allowing network defenders to neutralize the threat by blocking one address. Unlike advanced malware that uses resilient techniques like domain generation algorithms (DGAs), this sample's simplistic approach suggests it was designed for a specific, short-term operation or by an actor with an intermediate level of tradecraft.

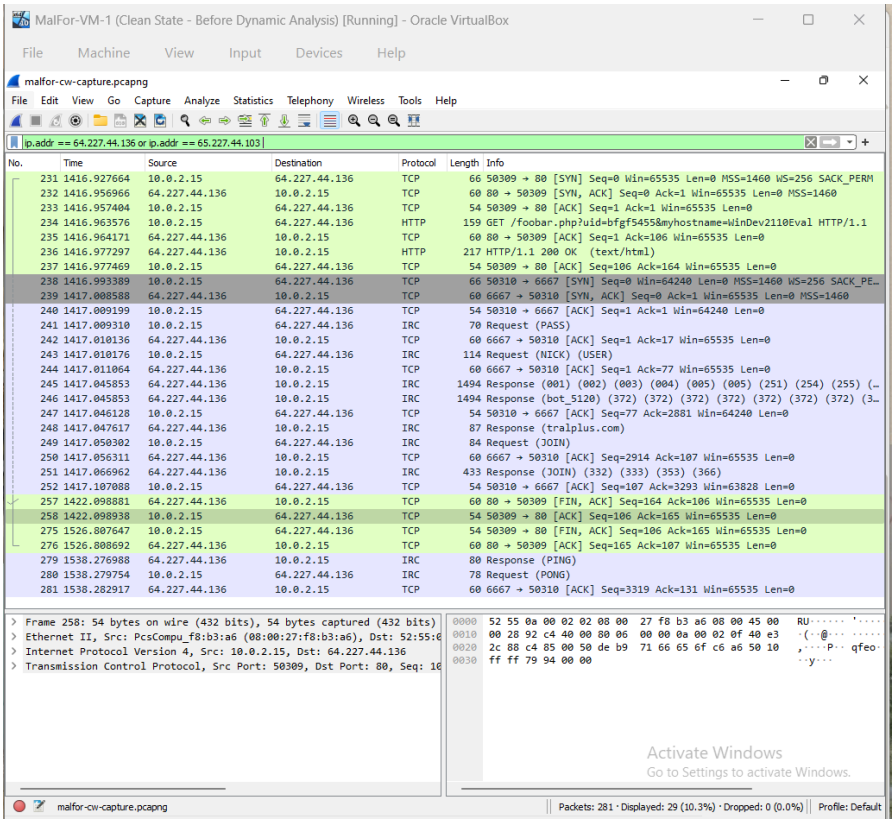
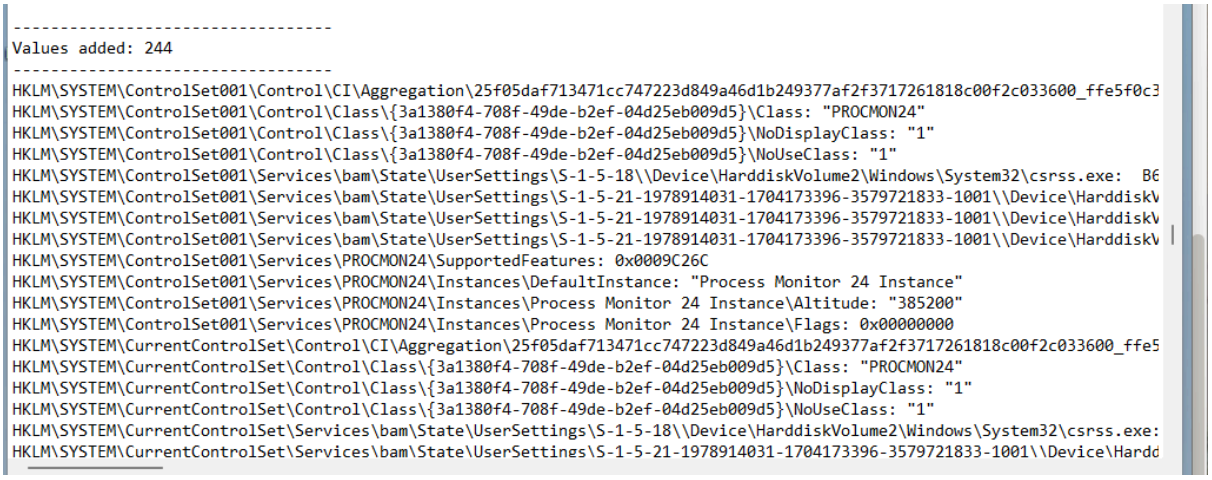


Figure 6: Wireshark capture showing IRC protocol commands being sent to the C2 server at 64.227.44.136 on the standard IRC port 6667.

Captured network data showing IRC commands used by the malware to join its C2 channel in Figure 6. The choice of the IRC protocol on its standard port (6667) is a notable evasion technique. Unlike a custom protocol on an unusual port, IRC traffic can be more easily mistaken for legitimate activity in a busy network, thereby reducing the chances of immediate detection.

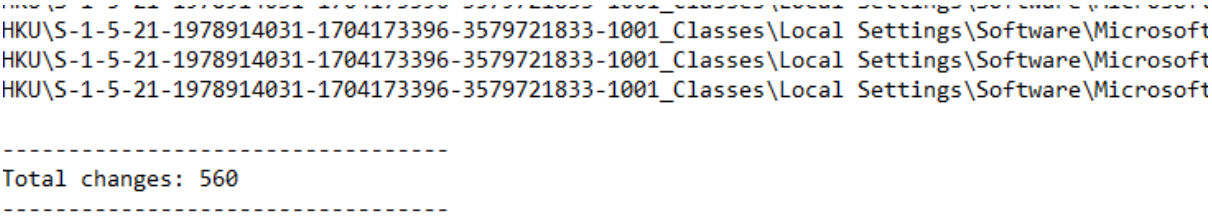
3.2 Persistence Mechanism Verification

Using RegShot, a snapshot of the registry was taken before and after running the malware. The comparison report confirmed that the malware creates a new value in SOFTWARE\Microsoft\Windows\CurrentVersion\Run. This action ensures the malware is automatically executed every time the system starts, validating the persistence hypothesis. The malware's ability to take screenshots and download additional files was also confirmed through commands identified during reverse engineering.



```
----------------------------------
Values added: 244
----------------------------------
HKLM\SYSTEM\ControlSet001\Control\CI\Aggregation\25f05daf713471cc747223d849a46d1b249377af2f3717261818c00f2c033600_ffe5f0c3
HKLM\SYSTEM\ControlSet001\Control\Class\{3a1380f4-708f-49de-b2ef-04d25eb009d5}\Class: "PROCMON24"
HKLM\SYSTEM\ControlSet001\Control\Class\{3a1380f4-708f-49de-b2ef-04d25eb009d5}\NoDisplayClass: "1"
HKLM\SYSTEM\ControlSet001\Control\Class\{3a1380f4-708f-49de-b2ef-04d25eb009d5}\NoUseClass: "1"
HKLM\SYSTEM\ControlSet001\Services\bam\State\UserSettings\S-1-5-18\\Device\HarddiskVolume2\Windows\System32\csrss.exe:  B6
HKLM\SYSTEM\ControlSet001\Services\bam\State\UserSettings\S-1-5-21-1978914031-1704173396-3579721833-1001\\Device\HarddiskV
HKLM\SYSTEM\ControlSet001\Services\bam\State\UserSettings\S-1-5-21-1978914031-1704173396-3579721833-1001\\Device\HarddiskV
HKLM\SYSTEM\ControlSet001\Services\bam\State\UserSettings\S-1-5-21-1978914031-1704173396-3579721833-1001\\Device\HarddiskV
HKLM\SYSTEM\ControlSet001\Services\PROCMON24\SupportedFeatures: 0x0009C26C
HKLM\SYSTEM\ControlSet001\Services\PROCMON24\Instances\DefaultInstance: "Process Monitor 24 Instance"
HKLM\SYSTEM\ControlSet001\Services\PROCMON24\Instances\Process Monitor 24 Instance\Altitude: "385200"
HKLM\SYSTEM\ControlSet001\Services\PROCMON24\Instances\Process Monitor 24 Instance\Flags: 0x00000000
HKLM\SYSTEM\CurrentControlSet\Control\CI\Aggregation\25f05daf713471cc747223d849a46d1b249377af2f3717261818c00f2c033600_ffe5
HKLM\SYSTEM\CurrentControlSet\Control\Class\{3a1380f4-708f-49de-b2ef-04d25eb009d5}\Class: "PROCMON24"
HKLM\SYSTEM\CurrentControlSet\Control\Class\{3a1380f4-708f-49de-b2ef-04d25eb009d5}\NoDisplayClass: "1"
HKLM\SYSTEM\CurrentControlSet\Control\Class\{3a1380f4-708f-49de-b2ef-04d25eb009d5}\NoUseClass: "1"
HKLM\SYSTEM\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-18\\Device\HarddiskVolume2\Windows\System32\csrss.exe:
HKLM\SYSTEM\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-21-1978914031-1704173396-3579721833-1001\\Device\Hardd
```

Figure 7: RegShot comparison shows values added by malware.



```
HKU\S-1-5-21-1978914031-1704173396-3579721833-1001_Classes\Local Settings\Software\Microsoft
HKU\S-1-5-21-1978914031-1704173396-3579721833-1001_Classes\Local Settings\Software\Microsoft
HKU\S-1-5-21-1978914031-1704173396-3579721833-1001_Classes\Local Settings\Software\Microsoft

----------------------------------
Total changes: 560
----------------------------------
```

Figure 8: RegShot comparison showing final number of changes caused by the malware.

This dynamic result provides definitive validation for the hypothesis formed during static analysis. The malware does not just contain strings related to persistence; it actively and successfully modifies the registry to survive a system reboot.

4. Reverse Engineering

Reverse engineering the unpacked executable with IDA Pro provided definitive proof of the malware's functionality by mapping its observed behaviours to specific Windows API calls. This analysis moves from what the malware does to how it does it. Key functions were identified that enable persistence, C2 communication, spyware, and anti-analysis tactics. For example, the call to IsDebuggerPresent (Figure 12) confirms the malware's attempt to evade analysis. The core capabilities are correlated with their enabling functions in Table 2.

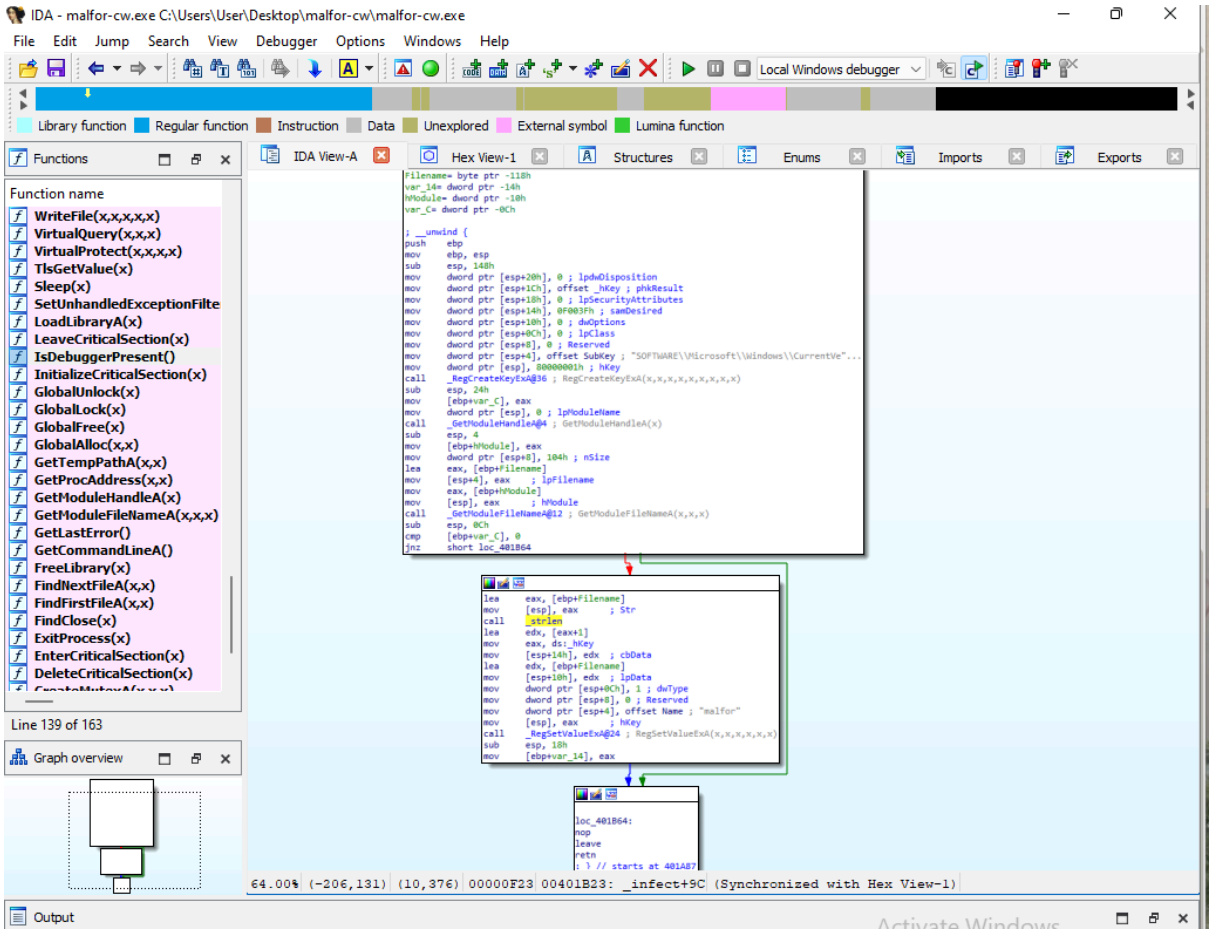


Figure 9: The call to RegSetValueExA responsible for establishing persistence in the Windows Registry.

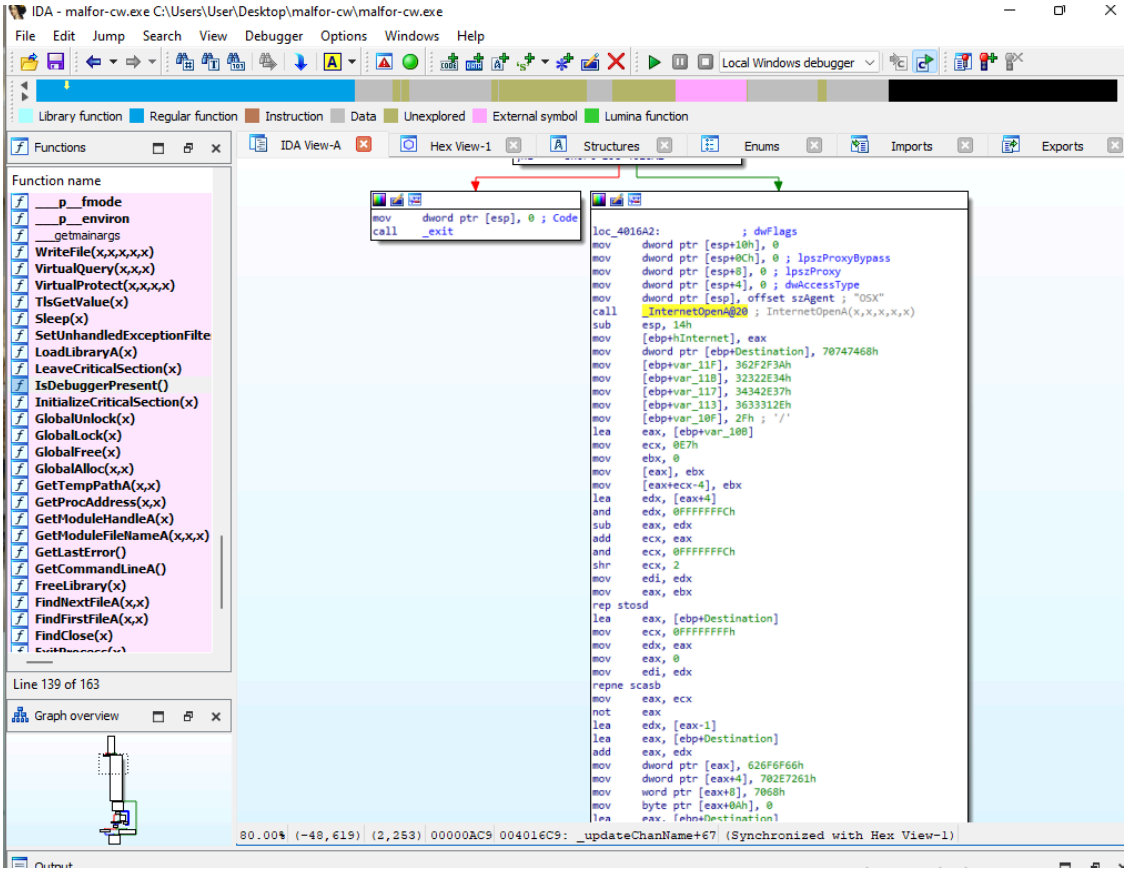


Figure 10: The InternetOpenA API call, which prepares the malware to communicate with the C2 server.

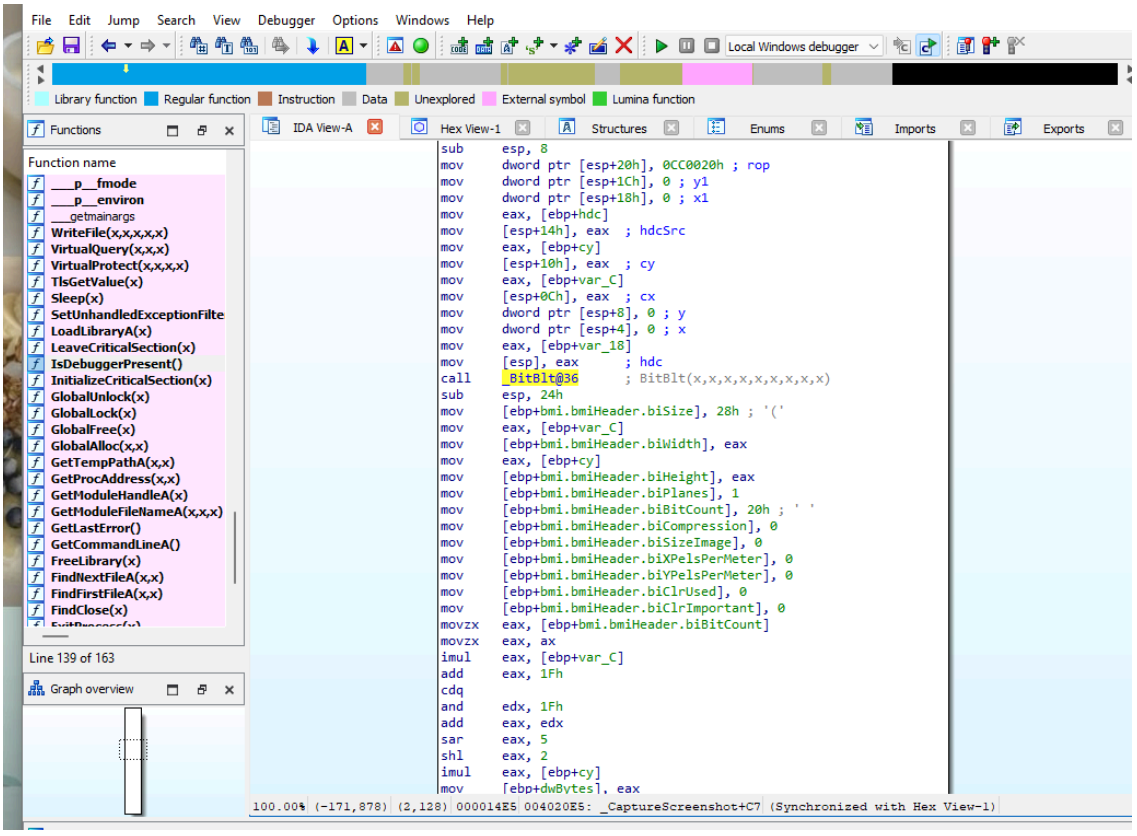


Figure 11: The BitBlt GDI function call, which enables the malware's screen capture (spyware) capability.

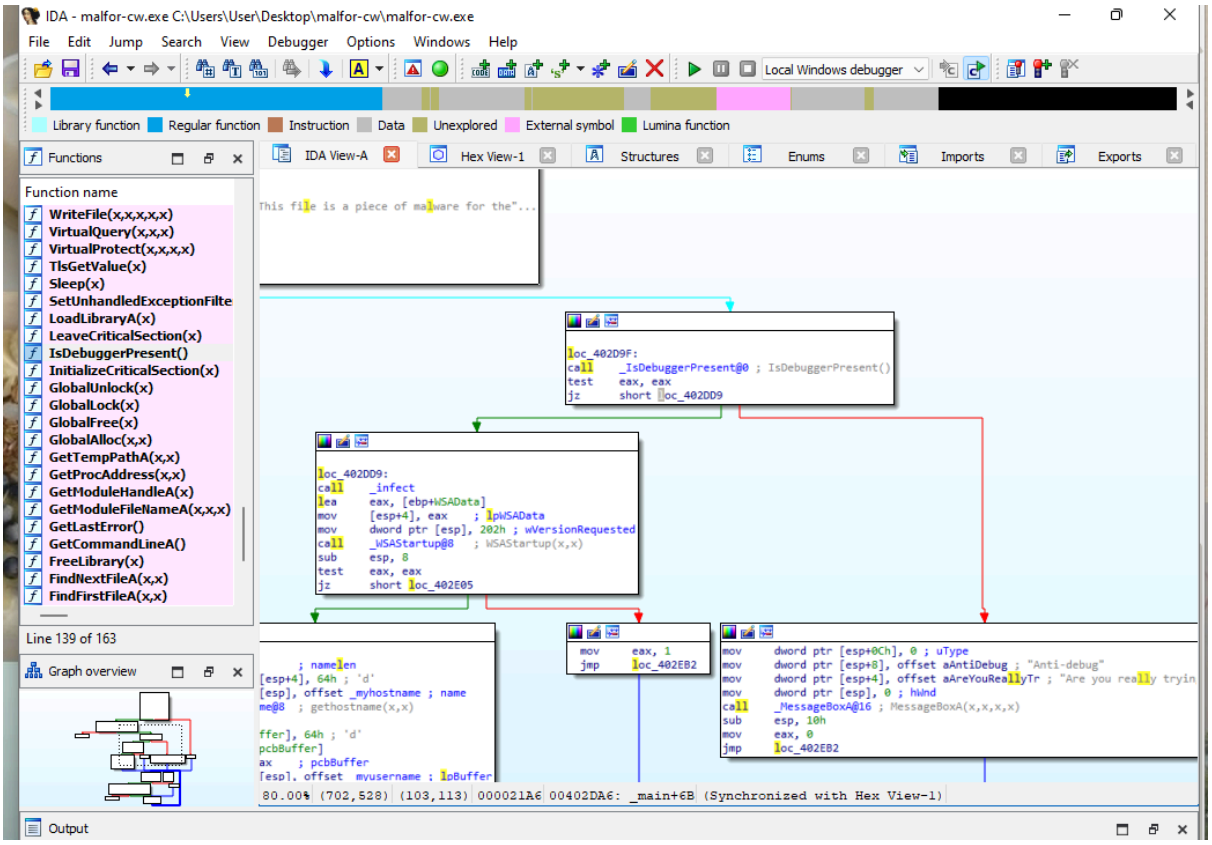


Figure 12: Disassembly view in IDA Pro showing the call to the IsDebuggerPresent API, a common anti-debugging technique.

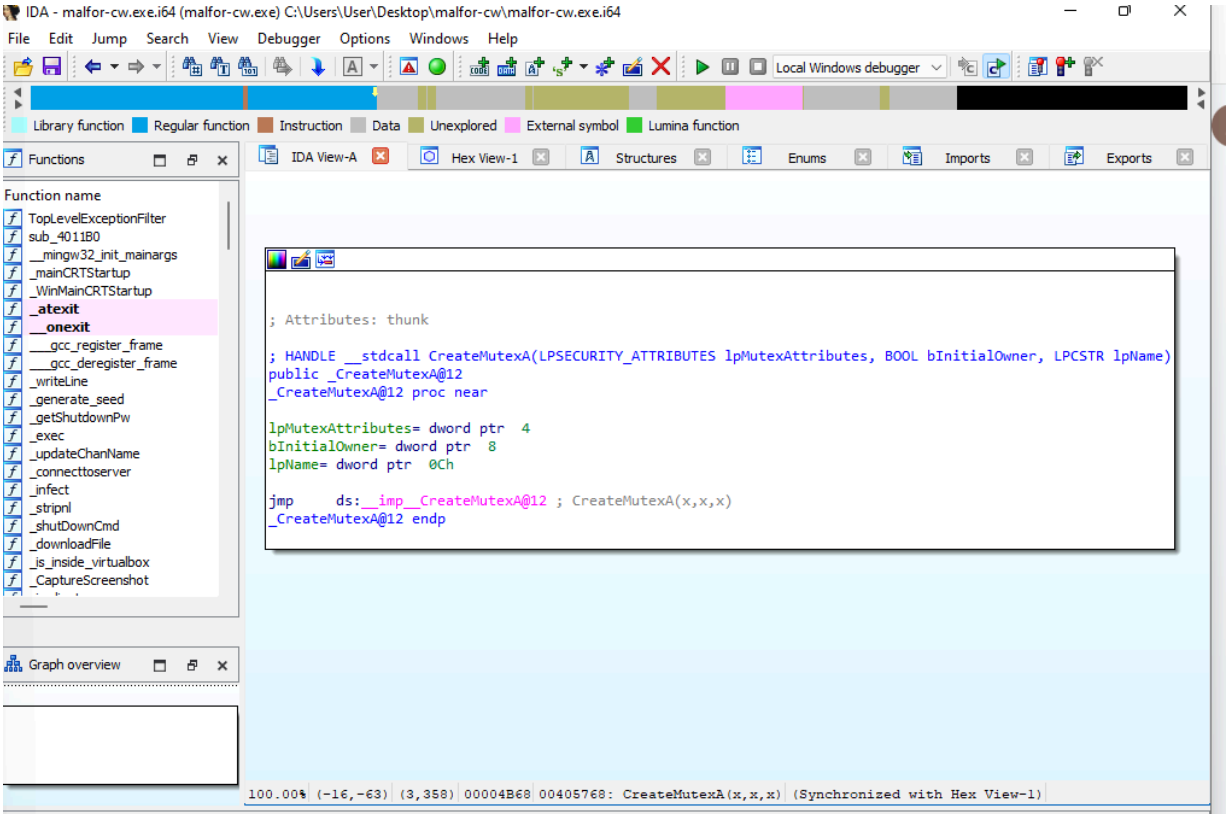


Figure 13: The CreateMutexA call, used to ensure only one instance of the malware runs at a time.

Table 2: Correlation of Malware Capabilities with API Functions

Capability	API Functions	Analysis
Persistence	RegSetValueExA	The malware writes its file path to the …\CurrentVersion\Run key using the RegSetValueExA function {}, ensuring it starts automatically with windows.
C2 Communication	InternetOpenA, connect	Network connections to the C” server are established by calling InternetOpenA and connect from, WININET.DLL and WS2_32.dll.
SpyWare	BitBlt, CreateCompatibleBitmap	The presence of these GDI32.dll functions confirms the capability to capture the screen, as suggested by the “screenshot.bmp” string
Anti-Debugging	IsDebuggerPresent	To evade analysis, the program checks if its being run under a debugger by calling IsDebuggerPresent
SIngle instance	CreateMutexA	The malware calls CreateMUtexA to ensure only one instance is running to prevent multiple executions in sandbox environment.

The reverse engineering phase confirms the attacker’s methodology. The call to IsDebuggerPresent is a simple but effective first-line defense against analysis, designed to thwart automated sandbox environments. The use of CreateMutexA is a more subtle anti-analysis technique; beyond preventing multiple instances, it can also be used to signal to itself that it has already been executed, which can complicate analysis in environments that run a sample multiple times. These checks demonstrate that the malware author had a clear intent to make investigation more difficult.

5. Discussion

5.1 Summary of Findings

The comprehensive analysis confirms that malfor-cw.exe is an IRC-based bot designed for espionage and remote control. It achieves persistence through a standard registry autorun key, communicates with a hardcoded C2 server over IRC, and possesses spyware capabilities, including screen capture. Furthermore, it employs basic anti-analysis techniques such as VM and debugger detection to hinder investigation. The combination of these features makes it a potent tool for an insider threat.

5.2 Threat Assessment for Gasgo
For the client, Gasgo, this malware represents a critical security breach. The screenshot functionality, confirmed by the BitBlt API call and the "screenshot.bmp" string, is the likely mechanism used by the rogue employee to steal the sensitive gas pipeline plans. The malware's use of a common protocol like IRC for C2 allows its traffic to be potentially overlooked in a busy network.

The most significant ongoing threat is the malwares ability to receive and execute commands, including the DOWNLOAD command. This means the attacker could have used the initial foothold to deploy more advanced malware, such as ransomware, keyloggers, or tools for lateral movement across Gasgos network. The breach should not be considered contained until all infected systems are identified and remediated.

5.3. Recommendations

To mitigate this threat and strengthen security posture, the following actions are recommended for immediate implementation:

Indicators of Compromise (IOCs):

- File Hash (MD5): f3c497aef3e476eaba09c470a9e9c22
- File Hash (SHA256): 258b7e2432cdfbc219e81709dc61212e6ea50a136a2c93ad2454f79e274f5
- IP Address: 64.227.44.136
- Domain: harryhill.com
- Registry Key: HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- Mutex Name:

Remediation Steps:

- Network Containment: Block all outbound traffic to IP address 64.227.44.136 and domain harryhill.com at the perimeter firewall.
- Host-Based Identification: Scan the entire network for files matching the provided hashes and for systems with the specified registry key modification.
- Eradication: On each infected machine:
    - Disconnect the machine from the network.
    - Open Task Manager and terminate the malfor-cw.exe process.
    - Delete the executable from its location on the file system.

    - Open the Registry Editor (regedit) and remove the malicious value from the ...\CurrentVersion\Run key.
    - Perform a full antivirus scan.

Security Hardening:

- Egress Filtering: Implement stricter firewall rules to block non-standard protocols like IRC on ports other than those required for legitimate business functions.
- Endpoint Detection and Response (EDR): Deploy an EDR solution configured to alert on common persistence techniques, such as modifications to the Run keys.
- User Training: Reinforce security awareness training, particularly regarding the risks of insider threats and running unauthorised software.

The true threat of this malware lies in its simplicity and effectiveness. For the client, Gasgo, the BitBlt screen capture function is the likely vector for espionage, allowing a rogue insider to exfiltrate sensitive data with minimal technical effort. However, the threat extends beyond data theft. An established IRC bot on a corporate network serves as a persistent foothold. The attacker could leverage this access to download and execute more damaging payloads, such as ransomware or keyloggers, or use it as a pivot point for lateral movement across the network. Therefore, the incident response should not only focus on this specific threat but also on the possibility that it was a precursor to a wider compromise.

In conclusion, the analysis of malfor-cw.exe demonstrates that a security incident's impact is not solely determined by the technical sophistication of the malware involved. While this IRC bot utilized common and detectable techniques, its deployment by a trusted insider allowed it to bypass perimeter defenses and achieve its objective. This investigation serves as a critical reminder for Gasgo that a truly resilient security posture must be multi-layered, integrating the technical controls recommended above with a robust insider threat program and a security-aware culture.

6. References

- Sikorski, M., & Honig, A. (2012). *Practical malware analysis: The hands-on guide to dissecting malicious software*. No Starch Press.
- Eagle, C. (2011). *The IDA Pro book: The unofficial guide to the world's most popular disassembler* (2nd ed.). No Starch Press.

7. Appendices

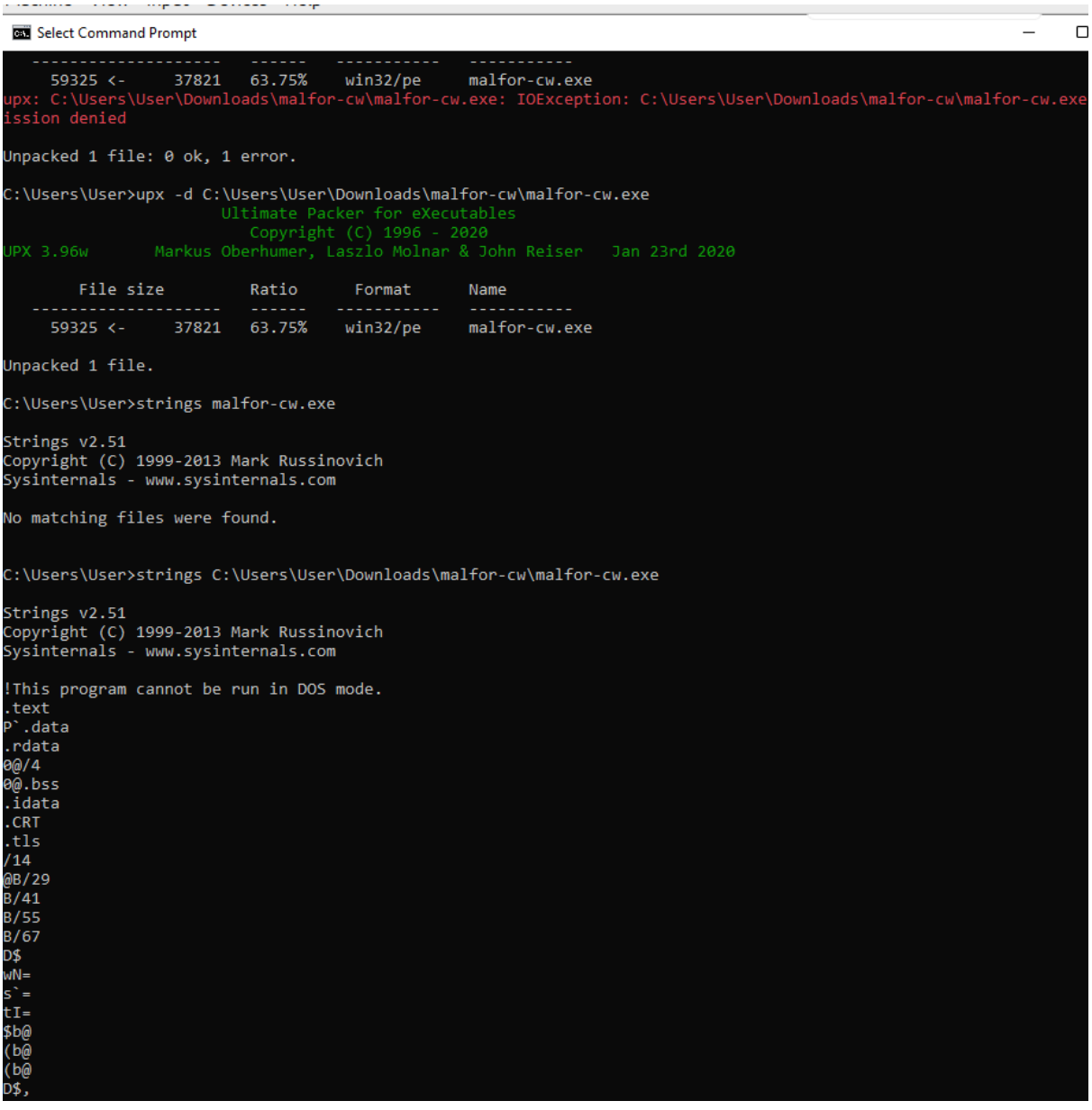


Figure A1: Command line output showing the successful unpacking of malfor-cw.exe using UPX, and strings outcome.

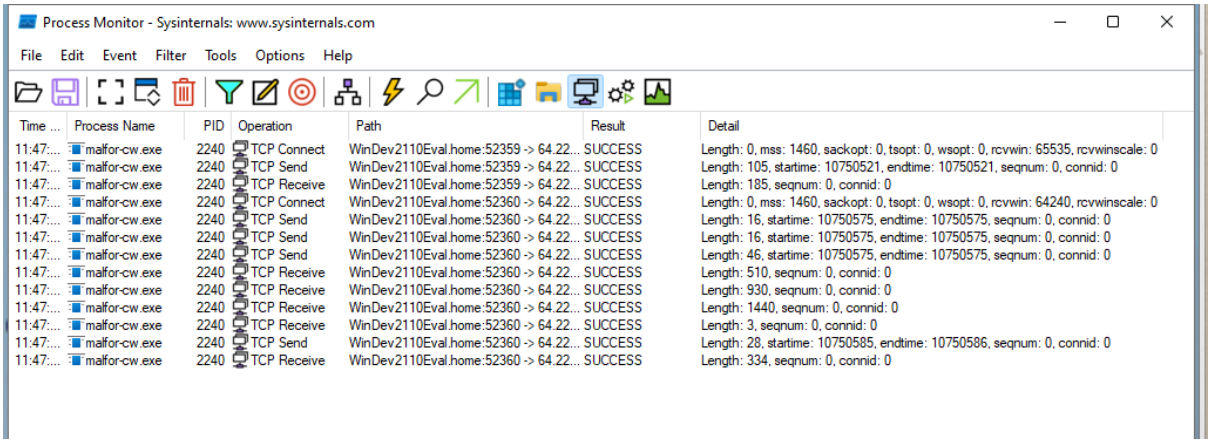


Figure A2: Process Monitor log detailing the initial events upon execution of malfor-cw.exe.

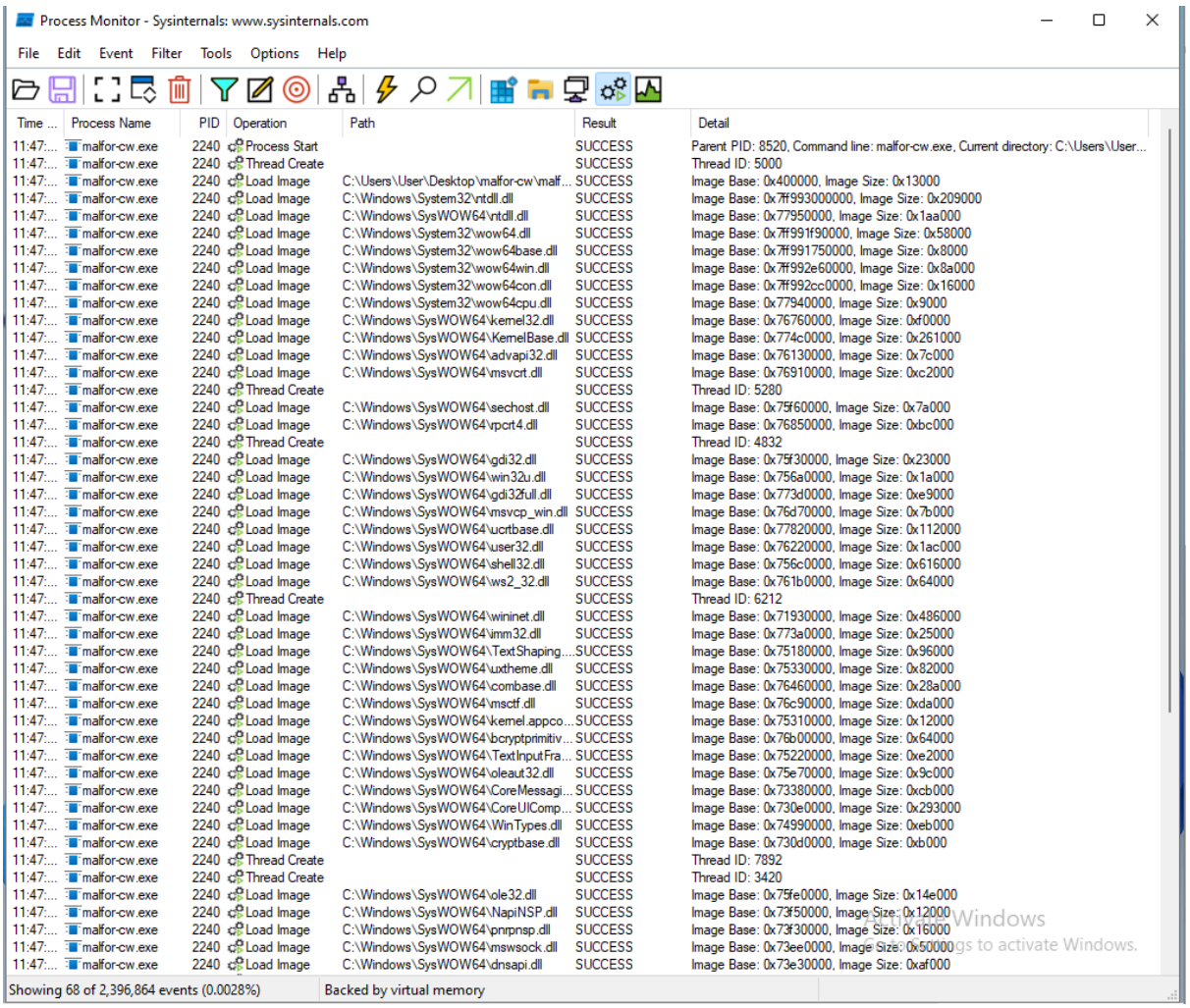
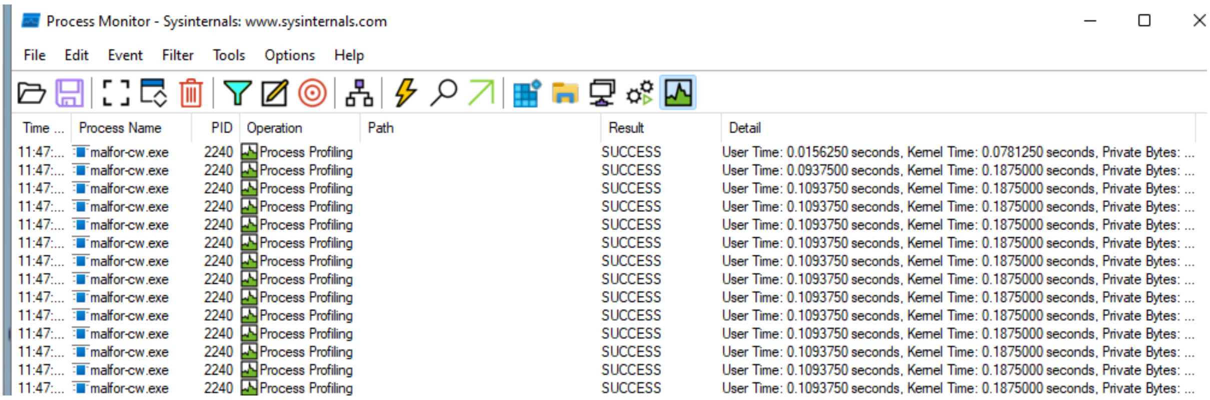


Figure A3: Full list of Dynamic-Link Libraries (DLLs) loaded by the malware at runtime, as captured by Process Monitor.



Time ...	Process Name	PID	Operation	Path	Result	Detail
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.0156250 seconds, Kernel Time: 0.0781250 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.0937500 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...
11:47:...	malfor-cw.exe	2240	Process Profiling		SUCCESS	User Time: 0.1093750 seconds, Kernel Time: 0.1875000 seconds, Private Bytes: ...

Figure A4: Process Monitor log showing Process Profiling events for the malfor-cw.exe process. This view captures performance metrics such as CPU time (User and Kernel) and memory usage over the execution lifetime.